

Теоретический материал: Функции и область видимости

1. Что такое функция?

Функция - это именованный блок кода, который выполняет конкретную задачу. Мы пишем его один раз, а потом вызываем столько раз, сколько нужно.

- **Определение функции** начинается с ключевого слова *def*.
- **Тело функции** - это код с отступом под *def*.

Создание функции в Python всегда следует строгому шаблону.

```
def имя_функции(параметры):  
    # Тело функции (код)  
    # Здесь выполняются действия  
    return результат # Необязательно
```

Разбор ключевых слов:

1. **def** (от англ. *define*) - сигнальная ракета для Python: «Внимание, сейчас я создам функцию!».
2. **Имя функции** - как мы будем к ней обращаться. Пишется маленькими буквами, слова разделяются подчеркиванием (например, `calculate_total`).
3. **Параметры (аргументы)** - это «входные отверстия» функции. Сюда мы передаем данные, с которыми функция будет работать. Если данные не нужны, скобки остаются пустыми: `()`.
4. **Двоеточие :** - обязательный знак в конце строки объявления.
5. **Отступ (4 пробела)** - всё, что написано с отступом, считается «внутренним миром» функции (её телом).
6. **return** - команда, которая «выбрасывает» результат работы функции наружу.

Простой пример создания

Она просто выполняет одно и то же действие при каждом вызове.

```
def say_welcome():  
    print("Добро пожаловать в систему Kaspi!")  
    print("Пожалуйста, авторизуйтесь.")  
  
# Вызов функции:  
say_welcome()
```

2. Глобальные переменные (Global)

Переменные, созданные **вне** любых функций, называются **глобальными**.

- Они "живут" во всем файле.
- Их можно читать из любой части кода, включая функции.
- **Аналогия:** Глобальная переменная - это как уличное освещение. Его видят все жители города.

3. Локальные переменные (Local)

Переменные, созданные **внутри** функции, называются **локальными**.

- Они "живут" только пока работает функция.
- Как только функция завершилась, локальная переменная удаляется из памяти.
- **Аналогия:** Локальная переменная - это светильник в вашей квартире. Соседи с улицы его не видят, и вы не можете включить его, находясь в другом районе.

Примеры кода

Пример 1: Доступ к глобальной переменной

```
city = "Astana" # Глобальная переменная

def say_hello():
    print(f"Привет из города {city}!") # Функция видит
    глобальную переменную

say_hello() # Выведет: Привет из города Astana!
```

Пример 2: Ошибка доступа к локальной переменной

```
def my_home():
    lamp = "Настольная лампа" # Локальная переменная
    print(lamp)

my_home()
print(lamp) # ОШИБКА! Вне функции переменной lamp не
существует.
```

Пример 3: Одноименные переменные

Если создать внутри функции переменную с тем же именем, что и глобальная, Python создаст **новую локальную** переменную. Глобальная при этом не изменится.

```
x = 10 # Глобальный x

def change():
    x = 20 # Локальный x (не меняет глобальный!)
    print(f"Внутри функции x = {x}")

change()
print(f"Вне функции x = {x}") # Выведет 10
```

⚠ Ключевое слово **global**

Если вам **обязательно** нужно изменить глобальную переменную прямо из функции, используется слово **global**. Но помните: в хорошем коде этим пользуются редко, так как это запутывает программу.

```
score = 0

def win():
    global score # Говорим Python: "Используй глобальную
    переменную, не создавай новую"
    score += 10

win()
print(score) # Теперь выведет 10
```

10 практических задач для закрепления

Задача 1: Создайте глобальную переменную `name`. Напишите функцию, которая просто выводит это имя на экран.

Задача 2: Внутри функции создайте переменную `secret = "Пароль123"`. Попробуйте вывести её на экран **вне** функции и объясните результат (словесно).

Задача 3: Создайте глобальную переменную `balance = 1000`. Напишите функцию, которая прибавляет к этому балансу 500, используя ключевое слово `global`.

Задача 4: Напишите функцию, которая принимает один аргумент (число) и выводит его квадрат. Будет ли этот аргумент доступен вне функции?

Задача 5: Что выведет этот код? (Попробуйте ответить без запуска):

```
a = 5
def test():
    a = 10
test()
print(a)
```

Задача 6: Создайте две функции. В каждой из них создайте локальную переменную с одинаковым именем `x`. Будут ли они конфликтовать между собой?

Задача 7: Напишите программу, где есть глобальная переменная `limit = 100`. Создайте функцию, которая проверяет, больше ли переданное ей число, чем `limit`.

Задача 8: Создайте глобальную переменную `counter = 0`. Напишите функцию `reset()`, которая обнуляет `counter` с помощью `global`, и функцию `status()`, которая просто печатает его значение.

Задача 9: Попробуйте изменить глобальную переменную внутри функции без слова `global` (например, `x = x + 1`). Какую ошибку выдаст Python? (Это полезно для запоминания типа ошибки).